Errand Crew: măsuri de securitate

Oct 23, 2026 · @arthur

Principiul de bază

Nu ne bazăm pe faptul că modelul „nu se lasă păcălit”, ci limităm ce poate face agentul chiar dacă e păcălit.

Un exemplu real: pagina Devpost a hackathonului conținea o linie care cere repo privat și cinci colaboratori GitHub. Un agent de browser poate întâlni text de acest fel pe orice pagină.

1. Prompt injection la agenții de browser

Riscul principal e un text de pe o pagină web care încearcă să dea instrucțiuni agentului. Apărarea stă în design, nu în prompt.

- **Privilegii minime pentru workeri.** Un worker poate doar să navigheze și să citească: fără email, fișiere, plăți, shell sau acces la cheile noastre.
- **Allowlist de domenii aplicat în cod**, nu în prompt: URL-ul se verifică înainte de `goto`, iar request-urile în afara registrului se blochează în Playwright.
- **Utilizatorul nu introduce URL-uri.** Planner-ul alege doar din registrul de site-uri, ceea ce elimină și SSRF-ul.
- **Doar citire.** Fără login, fără trimis formulare, fără „cumpără”. Scriem asta explicit în README.
- **Ieșire strictă.** Workerul returnează JSON validat cu o schemă Pydantic (preț, nume, URL, lungimi maxime). Textul brut al paginii nu ajunge la judge sau la planner.
- **Separare de roluri.** Instrucțiunile workerului vin din registru, iar conținutul paginii e marcat ca date. Planul nu se schimbă după ce workerii pornesc.
- **Limite de execuție:** număr maxim de pași și timeout per worker, număr maxim de workeri per cerere.

Allowlist-ul nu oprește o injecție pe un site permis, de aceea contează restul. În submit nu scriem „am rezolvat prompt injection”, ci că riscul e limitat prin design.

2. Serverul MCP public

Serverul de pe Render e accesibil oricui are URL-ul, iar în spate sunt creditele Kernel și API-ul LLM. Este cel mai urgent punct practic.

- **Autentificare cu bearer token** pe endpoint-ul MCP.
- **Validare `Origin`**, recomandată de spec pentru Streamable HTTP, contra DNS rebinding.
- **Rate limit și plafon de concurență** (de exemplu maximum 2 errands simultane), plus **plafon de cost per errand**.
- **`errand_id` ca UUID**, imposibil de ghicit. Rezultatele se citesc doar cu el.
- **Sesiunile Kernel se închid mereu** la final, chiar și la eroare (bloc `finally`).

3. Secrete și repo public

O cheie comisă o dată rămâne în istoricul Git chiar dacă o ștergi după, iar repo-ul va fi public.

- Cheile stau doar în variabile de mediu pe Render și într-un `.env` local, cu `.env` în `.gitignore`. Nu se trimit în chat.
- **Înainte de a face repo-ul public**, rulăm un scan de secrete (gitleaks) pe tot istoricul.
- După hackathon, rotim cheile.

4. Demo-ul web

Rezultatele vin de pe site-uri externe și ajung într-o pagină web, deci le afișăm ca text (`textContent`), nu ca HTML (`innerHTML`). Așa evităm XSS prin conținut extras.

5. Mărunțișuri

- Versiuni fixate în `requirements.txt`.
- Fără date personale în logs și în demo.

Ce includem în schelet din prima

Tot ce e mai sus în afară de rotirea cheilor intră în schelet, în aproximativ o oră de lucru: bearer token, validare Origin, rate limit și plafon de concurență, allowlist în cod, scheme Pydantic și închiderea sesiunilor Kernel.

Adăugăm și o secțiune „Security” în README, pe care judecătorii o pot verifica în repo. Contează la Tech Implementation.